

NET2008 Midterm: Sample Questions

1. You want to see who last changed each line of `ntp.cfg`. Which command do you run?

- A. `git diff ntp.cfg`
- B. `git status ntp.cfg`
- C. `git log ntp.cfg`
- D. `git blame ntp.cfg`

2. You want to add a new line to the end of an existing log file without erasing what is already in it. Which mode do you pass to `open`?

```
f = open("app.log", "_____")
```

- A. `"r"`
- B. `"a"`
- C. `"x"`
- D. `"w"`

3. What does this code print?

```
print(7 // 2, 7 % 2)
```

- A. `3.5 1`
- B. `4 1`
- C. `3 1`
- D. `3 0`

4. The code must print the numbers 1, 2 and 3, one per line. Which line fills the blank _____?

```
for i in _____:  
    print(i)
```

- A. `range(1, 3)`
- B. `range(3)`
- C. `range(4)`
- D. `range(1, 4)`

5. You are on `main` and want to create a branch called `fix-dns` and move onto it in one command. Which command do you run?

- A. `git switch -c fix-dns`
- B. `git branch fix-dns`
- C. `git merge fix-dns`
- D. `git switch fix-dns`

6. You are on a new computer and want a copy of a repository that exists only on GitHub. Which command do you run?

- A. `git init URL`
- B. `git pull URL`
- C. `git push URL`
- D. `git clone URL`

7. What does this code print?

```
import json
print(json.dumps({"up": True}))
```

- A. `{"up": true}`
- B. `{'up': true}`
- C. `up: true`
- D. `{"up": True}`

8. The branch `guest-vlan` exists on your computer and also on GitHub, where you pushed it earlier. On your computer you merge `guest-vlan` into `main`, then run `git branch -d guest-vlan`. Which statement is true?

- A. The branch is deleted on your computer and on GitHub.
- B. The merged commits are removed from your local `main`.
- C. The branch name is deleted on your computer only. `guest-vlan` still exists on GitHub, and the merged commits stay on your local `main`.
- D. Git refuses, because the branch still exists on GitHub.

9. What does this code print?

```
import json
print(json.dumps({"a": (1, 2), "b": None, "c": True, 3: "x"}))
```

- A. `{"a": [1, 2], "b": null, "c": true, 3: "x"}`
- B. `{"a": [1, 2], "b": null, "c": true, "3": "x"}`
- C. `{"a": [1, 2], "b": "None", "c": "True", "3": "x"}`
- D. `{"a": (1, 2), "b": None, "c": True, "3": "x"}`

10. The code must print `[22, 80, 443, 8080]`. Which method name fills the blank _____?

```
ports = [22, 80, 443]
ports.____(8080)
print(ports)
```

- A. `add`
- B. `extend`
- C. `append`
- D. `push`

11. The code below follows the PEP 8 naming conventions taught in the course everywhere except in one name. Which name breaks the convention?

```
DEFAULT_PORT = 22

class DeviceProfile:
    def __init__(self, name):
        self._name = name

    def checkServer(self):
        return self._name
```

- A. `checkServer`
- B. `DeviceProfile`
- C. `DEFAULT_PORT`
- D. `_name`

12. You are on `main`. You ran `git switch -c ntp-fix`, changed line 2 of `ntp.cfg` and committed. You ran `git switch main`, changed the same line 2 in a different way and committed. Now you run `git merge ntp-fix`. What happens?

- A. Git deletes `ntp-fix` and leaves `main` unchanged.
- B. Git keeps the line from the newer commit and finishes the merge silently.
- C. The merge stops with a conflict in `ntp.cfg`. The file contains conflict markers, and the merge is finished only after you edit the file, run `git add ntp.cfg` and run `git commit`.
- D. Git keeps both versions of the line and finishes the merge automatically.

13. What does this code print?

```
try:
    d = {"a": 1}
    print(d["b"])
except Exception:
    print("general")
except KeyError:
    print("key")
```

- A. It crashes with an unhandled `KeyError`.
- B. `general`
- C. `key`
- D. `general` and then `key`

14. What does this code print?

```
class DeviceError(Exception):
    pass

def poll(ip):
    try:
        if ip == "":
            raise ValueError("empty")
    except ValueError as e:
        raise DeviceError("poll failed: " + str(e))

try:
    poll("")
except DeviceError as e:
    print(e)
```

- A. Nothing; the exception is not caught.
- B. `poll failed: empty`
- C. `DeviceError: poll failed: empty`
- D. `empty`

15. A server is down. It answers with the status `503 Service Unavailable` and the HTML body `<h1>Service Unavailable</h1>` within one second. Your script runs the two lines below. What happens?

```
r = requests.get(url, timeout=5)
data = r.json()
```

- A. The first line raises `Timeout` because the server did not give the data.
- B. The first line raises `HTTPError` because the status is 503.
- C. The first line returns normally and `r.json()` returns an empty dictionary.
- D. The first line returns normally with `r.status_code` 503. The second line raises a JSON decoding error because the body is not JSON.

Questions 16 to 20 use the program below. Assume that the program has run completely, unless a question says otherwise.

backup.py

```
class BackupError(Exception):
    pass

class Config:
    def __init__(self, host):
        self.host = host
        self.values = {}

    def set(self, key, value):
        self.values[key] = value

def parse(text):
    data = {}
    for line in text.splitlines():
        if line.startswith("#"):
            continue
        key, value = line.split("=")
        data[key] = value
    return data

def save(cfg, saved):
    if cfg.host in saved:
        raise BackupError(cfg.host + " exists")
    saved.append(cfg.host)

text = """# core router
hostname=core-rtr
ntp=10.0.0.10"""
cfg = Config("core-rtr")
for k, v in parse(text).items():
    cfg.set(k, v)
```

16. How many items does `cfg.values` hold after the program has run?

- A. 0
- B. 3
- C. 2
- D. 1

17. What does `print(cfg.values["ntp"])` print after the program has run?

- A. `ntp`
- B. `10.0.0.10`
- C. `KeyError`
- D. `ntp=10.0.0.10`

18. What happens when the program calls `parse("vlan=10=20")`?

- A. It returns `{'vlan': '20'}`.
- B. It returns `{'vlan': '10=20'}`.
- C. It returns `{'vlan': '10'}`.
- D. It raises `ValueError`.

19. The program runs the code below after the lines shown. What does it print?

```
saved = []
try:
    save(cfg, saved)
    save(cfg, saved)
except BackupError as e:
    print(e)
```

- A. `core-rtr exists`
- B. `BackupError`
- C. `core-rtr`
- D. Nothing; the exception is not caught.

20. The program runs the code below after the lines shown. What does it print?

```
saved = []
save(cfg, saved)
save(Config("sw1"), saved)
print(saved)
```

- A. `['core-rtr', 'sw1']`
- B. `[]`
- C. `['sw1']`
- D. `['core-rtr']`

Answers and explanations

No.	Answer	Explanation
1	D	Correct: D. <code>git blame</code> prints each line with the commit and author that last changed it. A is wrong. <code>git diff</code> shows uncommitted changes. B is wrong. <code>git status</code> shows file states, not line history. C is wrong. <code>git log</code> lists commits, not lines.
2	B	Correct: B. Mode <code>"a"</code> appends: it keeps the existing content and writes at the end. A is wrong. <code>"r"</code> only reads; writing raises an error. C is wrong. <code>"x"</code> creates a new file and fails if the file already exists. D is wrong. <code>"w"</code> empties the file first.
3	C	Correct: C. <code>//</code> is floor division: 7 divided by 2 is 3 with a remainder. <code>%</code> gives the remainder, which is 1. A is wrong. <code>3.5</code> is the result of <code>/</code>, not <code>//</code>. B is wrong. <code>//</code> rounds down, not up, so it gives 3. D is wrong. The remainder of 7 divided by 2 is 1, not 0.
4	D	Correct: D. <code>range(1, 4)</code> starts at 1 and stops before 4, so it gives 1, 2, 3. A is wrong. <code>range(1, 3)</code> stops before 3, so it gives 1, 2. B is wrong. <code>range(3)</code> gives 0, 1, 2. C is wrong. <code>range(4)</code> gives 0, 1, 2, 3.
5	A	Correct: A. <code>-c</code> creates the branch and <code>git switch</code> moves to it. B is wrong. <code>git branch fix-dns</code> creates the branch but leaves you on <code>main</code>. C is wrong. <code>git merge</code> combines an existing branch into the current one. D is wrong. Without <code>-c</code>, Git reports that the branch does not exist.
6	D	Correct: D. <code>git clone</code> downloads the whole repository, with its history, into a new folder and sets up the connection to GitHub. A is wrong. <code>git init</code> creates a new empty repository and does not download anything. B is wrong. <code>git pull</code> updates a repository you already have. There is no repository here yet. C is wrong. <code>git push</code> sends your commits to GitHub; it does not download.
7	A	Correct: A. <code>json.dumps</code> produces JSON text, which uses lowercase <code>true</code> and double quotes. B is wrong. JSON requires double quotes, not single quotes. C is wrong. That is YAML style, not JSON. D is wrong. <code>True</code> with a capital T is Python, not JSON.
8	C	Correct: C. <code>git branch -d</code> works on your local branches. A branch on GitHub is a separate copy and needs its own delete. Deleting a branch name never removes commits that are already part of <code>main</code>. A is wrong. <code>git branch -d</code> only touches your own repository. GitHub is changed only by pushing. B is wrong. The commits are part of <code>main</code> after the merge. Deleting the branch name does not touch them. D is wrong. Git allows the delete because the branch is already merged into <code>main</code>. The copy on GitHub is not checked.

No.	Answer	Explanation
9	B	Correct: B. JSON has no tuples, so <code>(1, 2)</code> becomes an array <code>[1, 2]</code>. <code>None</code> becomes <code>null</code> and <code>True</code> becomes <code>true</code>. JSON object keys are always strings, so the integer key 3 becomes <code>"3"</code>. A is wrong. JSON object keys must be strings, so the integer key is written as <code>"3"</code>. C is wrong. <code>None</code> and <code>True</code> are converted to the JSON values <code>null</code> and <code>true</code>, not to strings. D is wrong. That is Python syntax. <code>dumps</code> produces JSON syntax: an array, <code>null</code> and <code>true</code>.
10	C	Correct: C. <code>append</code> adds one item to the end of the list. The list changes in place, so <code>print(ports)</code> shows <code>[22, 80, 443, 8080]</code>. A is wrong. <code>add</code> is a method of sets, not lists. <code>ports.add(8080)</code> raises <code>AttributeError</code>. B is wrong. <code>extend</code> adds every item of an iterable. <code>8080</code> is an integer, so <code>ports.extend(8080)</code> raises <code>TypeError</code>. D is wrong. Python lists have no <code>push</code> method. <code>ports.push(8080)</code> raises <code>AttributeError</code>.
11	A	Correct: A. Functions and methods use <code>lower_snake_case</code>, so the method should be named <code>check_server</code>. <code>checkServer</code> is camelCase, which PEP 8 does not use for functions. B is wrong. A class uses <code>CapWords</code>, so <code>DeviceProfile</code> is correct. C is wrong. A constant uses <code>UPPER_SNAKE_CASE</code>, so <code>DEFAULT_PORT</code> is correct. D is wrong. A leading underscore marks a name as intended for internal use. It is allowed and is a normal PEP 8 convention.
12	C	Correct: C. Both branches changed the same line, so Git cannot choose. It stops, writes <code><<<<<<<</code>, <code>=====</code> and <code>>>>>>>></code> markers into the file and waits for you. A is wrong. A conflicting merge does not delete anything. The branch stays, and the merge is simply left unfinished. B is wrong. Git never picks a winner by date. When the same line differs, it stops and asks you. D is wrong. Git cannot decide how to combine two different versions of one line. You decide by editing the file.
13	B	Correct: B. <code>except</code> clauses are checked from top to bottom, and the first one that matches runs. <code>KeyError</code> is a kind of <code>Exception</code>, so the first clause matches and the <code>KeyError</code> clause is never reached. A is wrong. The exception is caught by <code>except Exception</code>, so nothing is left unhandled. C is wrong. The most specific clause does not win automatically. The first matching clause, <code>Exception</code>, runs. D is wrong. Only one <code>except</code> clause runs for one exception.
14	B	Correct: B. <code>poll</code> catches the <code>ValueError</code> and raises a new <code>DeviceError</code> whose message is <code>"poll failed: " + str(e)</code>. The caller catches the <code>DeviceError</code>, and <code>print(e)</code> prints its message. A is wrong. The <code>DeviceError</code> raised in <code>poll</code> is caught by <code>except DeviceError</code> in the caller. C is wrong. <code>print(e)</code> prints only the message of the exception, not its class name. D is wrong. <code>empty</code> is the message of the <code>ValueError</code>. The caller catches the <code>DeviceError</code>, which has a different message.

No.	Answer	Explanation
15	D	Correct: D. <code>requests</code> does not raise for 4xx or 5xx statuses unless you call <code>raise_for_status()</code>. The server answered within the timeout, so <code>get</code> returns a <code>Response</code>. The HTML body cannot be parsed as JSON, so <code>r.json()</code> raises a decoding error. A is wrong. A timeout happens only when the server does not answer in time. This server answered within one second. B is wrong. <code>HTTPError</code> is raised only by <code>raise_for_status()</code>. The script never calls it. C is wrong. <code>r.json()</code> does not invent data. It parses the body, and an HTML body is not valid JSON.
16	C	Correct: C. <code>parse</code> skips the line that starts with <code>#</code>. The other two lines each add one key: <code>hostname</code> and <code>ntp</code>. So <code>cfg.values</code> has two items. A is wrong. <code>cfg.set</code> is called for every item that <code>parse</code> returns, so the dictionary is not empty. B is wrong. 3 would count the comment line, but <code>parse</code> skips lines that start with <code>#</code>. D is wrong. Both remaining lines are stored, not only one of them.
17	B	Correct: B. <code>line.split("=")</code> cuts the line <code>ntp=10.0.0.10</code> into <code>ntp</code> and <code>10.0.0.10</code>. The first becomes the key and the second the value. A is wrong. <code>ntp</code> is the key. Indexing with the key returns the value. C is wrong. The key <code>ntp</code> exists, because the line has no spaces around the <code>=</code>. D is wrong. <code>split</code> removes the <code>=</code> and cuts the line in two. The key and the value are stored separately.
18	D	Correct: D. <code>split("=")</code> on <code>vlan=10=20</code> gives three parts: <code>vlan</code>, <code>10</code> and <code>20</code>. The line <code>key, value = ...</code> expects two, so Python raises <code>ValueError</code> (too many values to unpack). A is wrong. Nothing keeps only the last part. The unpacking fails, so no dictionary is returned at all. B is wrong. <code>split</code> does not stop at the first <code>=</code> unless it is told to. It makes three parts. C is wrong. Nothing is cut off after the first <code>=</code>. The line is split into three parts, and the unpacking fails before anything is stored.
19	A	Correct: A. The first <code>save</code> adds <code>core-rtr</code> to <code>saved</code>. The second <code>save</code> finds the host in <code>saved</code> and raises <code>BackupError("core-rtr exists")</code>. The <code>except</code> clause catches it, and <code>print(e)</code> prints its message. B is wrong. <code>print(e)</code> prints the message of the exception, not the name of its class. C is wrong. <code>core-rtr</code> is only the host. The message is the host plus the word <code>exists</code>. D is wrong. <code>except BackupError</code> matches the exception that <code>save</code> raises, so it is caught.
20	A	Correct: A. <code>save</code> changes the list it receives with <code>append</code>, so both calls add to the same <code>saved</code> list. The first adds <code>core-rtr</code> and the second adds <code>sw1</code>. B is wrong. <code>append</code> changes the original list, even though <code>save</code> does not return anything. C is wrong. The list is not replaced by the second call. It keeps what was already added. D is wrong. The second call also appends, because <code>sw1</code> is not in <code>saved</code> yet.